

Tree Based Models

Data 624 - Predictive Analytics

Rebecca Bronstein
Michael Drake
Randy Howk
Jayden Jiang

Decision Trees

What is a Decision Tree?

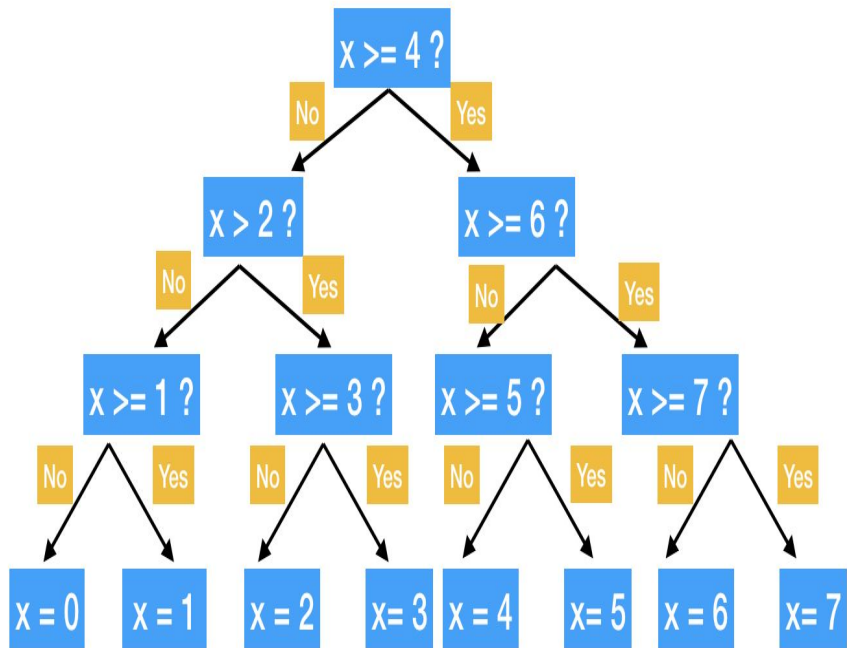
- Machine based learning algorithm to in organizing data in a visual manor decisions
- Used to split data into different categorizations based on similar properties into nodes or different regions
- CART (Classification and Regression Tree)
 - Tree Types
 - Categorization (mode)
 - Regression (mean)

Advantages of a Decision Tree

- Do not assume or require a predefined relationship between predictors and the response
- Require little to no data preprocessing and can naturally handle diverse predictor types (e.g., sparse, skewed, continuous, categorical)
- Are robust to multicollinearity among predictors
- Can effectively manage datasets with missing values
- Are easy to implement using widely available, user-friendly packages

How do Models Make Decisions?

- Start with Data point
- Starts at the top of the tree
- Goes down tree following each rule set by model
- End at a terminal node or leaf
- Output Each Value into leaf
- Each observation is one unique path



How Do Regression Based Trees Make Decisions? (Classification)

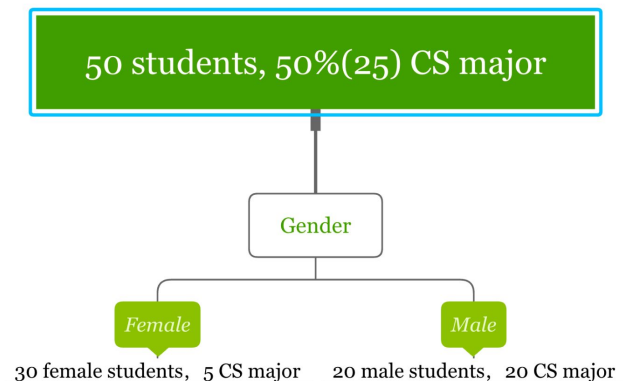
- Measure of non-homogeneity
- p_i is the probability of class i and the interval of Gini is $[0,0.5]$.
 - 0 is the smallest Gini score and 0.5 is the largest
- Helps decide the best split

$$Gini\ Index = \sum_i p_i(1 - p_i)$$

For a two-class problem

$$p_1(1 - p_1) + p_2(1 - p_2)$$

Categorization Tree Example



$$p_1(1 - p_1) + p_2(1 - p_2)$$

Dataset:

- 50 students, 50% (25) are CS majors

Split by Gender:

- Female (30 students, 5 CS)
Gini = $(1/6 \times 5/6) + (5/6 \times 1/6) = 5/18$
- Male (20 students, 20 CS)
Gini = $0 \times 1 + 1 \times 0 = 0$

Weighted Gini after split:

$$= (3/5 \times 5/18) + (2/5 \times 0) = 1/6$$

Parent node Gini:

$$= 1/2$$

Gini decrease:

$$= 1/2 - 1/6 = 1/3$$

Conclusion:

- The split on Gender significantly reduces impurity.
- Decision trees choose splits that maximize this decrease.

Entropy

Regression Trees

How do Regression Trees Models Make Decisions? (Regression)

- Data is grouped using the Sum of Squared Errors
- Most Similar Data is grouped together

Key Formula: Sum of Squared Errors (SSE)

$$SSE = \sum (y_i - \bar{y})^2$$

- y_i : actual value
- $\bar{y}$: average in that group

Regression Tree

- How to set up
 - Divide Predictors into non overlapping regions
 - $X_1, X_2, X_3 \dots$
 - When dividing limited RSS or Residual Sum of Squares for optimal fit
- For every observation that falls into the region R_j , the prediction is the mean of the response values for the training observations in R_j
- Advantages
 - Single tree easy to interpret
- Disadvantages
 - Low Accuracy
 - Unstable
 - Small Changes lead to very different trees

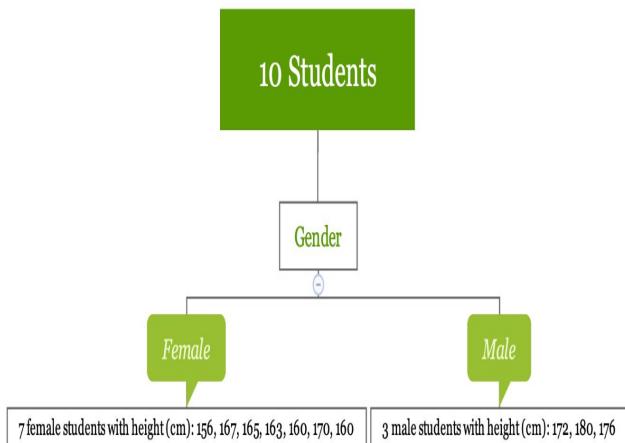
$$R_1(j, s) = \{X | X_j < s\} \text{ and } R_2(j, s) = \{X | X_j \geq s\}$$

Calculate the RSS decrease after the split. For different (j, s) , search for the combination that minimizes the RSS, that is to minimize the following:

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

where $\hat{y}_{R_1}$ is the mean of all samples in R_1 , $\hat{y}_{R_2}$ is the mean of samples in R_2 . It can be quick to optimize the equation above. Especially when p is not too large.

Regression Tree Example

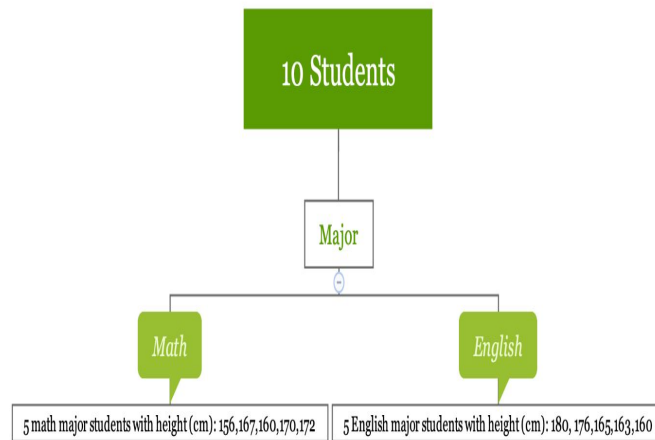


You can calculate the SSE using the following code:

1. SSE for "Female" is 136
2. SSE for "Male" is 32
3. SSE for splitting node "Gender" is the sum of the above two numbers which is 168

SSE for the 10 students in root node is 522.9. After the split, SSE decreases from 522.9 to 168.

If there is another possible way of splitting, divide it by major, as follows:



In this situation:

1. SSE for "Math" is 184
2. SSE for "English" is 302.8
3. SSE for splitting node "Major" is the sum of the above two numbers which is 486.8

Splitting data using variable "gender" reduced SSE from 522.9 to 168; using variable "major" reduced SSE from 522.9 to 486.8. Based on SSE reduction, you should use gender to split the data.

Tree Pruning

- Why?

- Limit Size
- Minimize Complexion Errors

- How?

- Limiting size of each node
- Maximizing size of depth
- Maximizing number of terminal nodes
- Change Number of Variables
 - Use the square root of numbers
- Complexity Formula
 - Remove insignificant branches

Complexity Formula

$$SSE_{\lambda} = SSE + \lambda \times (\textit{complexity})$$

Error Based Pruning

- All split nodes are included in initial candidate pool
- Pruning split node
 - Removing entire subtree under node and setting terminal node
- Subsets
 - training a complete tree
 - pruning
 - testing the final model

Decision Tree

- Similar to Regression
- Used for Categorical Data and or non-continuous Data
- Group categories on either side of split
 - Treats categorical predictor as ordered set
 - Encodes Categorical variables into binary
 - Allows for independent bits of information

Segment	One-hot encoding →	segmentConspicuous	segmentPrice	segmentQuality	segmentStyle
Conspicuous		1	0	0	0
Price		0	1	0	0
Quality		0	0	1	0
Style		0	0	0	1

FIGURE 11.3: One-hot encoding

Bagged Trees

Bagging Tree

- Bootstrapping

- Grabbing samples of same size from original to generate more data

- Advantages

- Stabilizes models by averaging results
 - More accurate predictions
 - Compared to regression or MARS might only be marginal
 - Can use out of bag samples to see how it affects models for predictive performance

- Disadvantages

- As the number of samples the need for better computing does too
 - Can be difficult to explain results
 - Uses original predictors for every tree, meaning that they are related
 - Can reduce variance

Random Forest

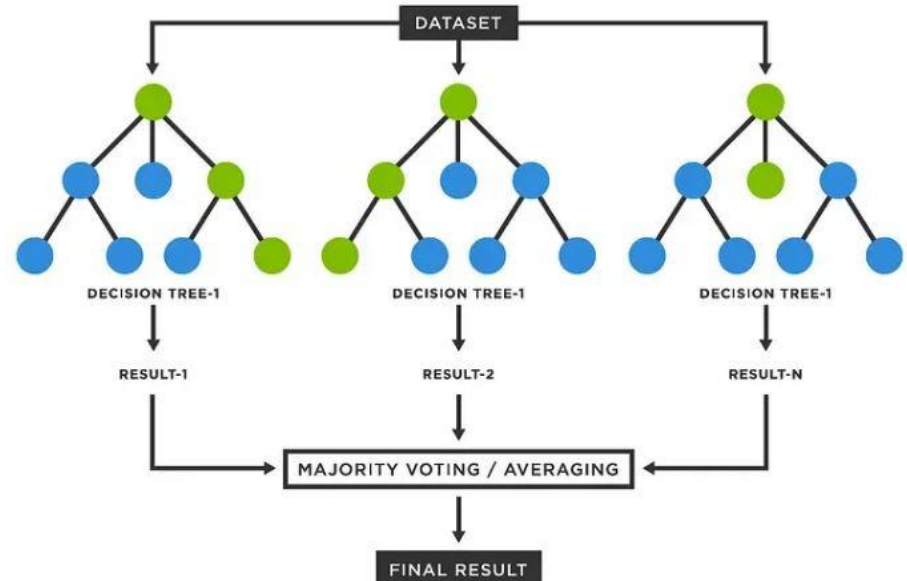
- Reduces correlation among trees to improve bagging predictive values
- Improves Randomness for more accurate results
- How to build?
 - Algorithm randomly chooses m to use for splitting node
 - Choose best M for best node
 - $m=\sqrt{p}$.
 - Ex. Using a sample of 10 choose 4 randomly
 - Can cause less correlation since not always using same samples as bagging
- Works better with more predictors
 - Can increase trees until performance levels off
- When $m=p$, random forest is equal to the bagging tree. When the predictors are highly correlated, then smaller m tends to work better.

Random Forests

“a machine learning model built from many decision trees instead of just one. In simple words, RF is like asking many trees for opinion instead of relying on only one”

Random Forest

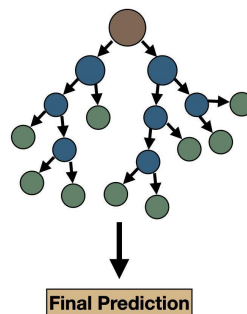
- An ensemble learning Method
- It builds many decision trees
- Combines their predictions
- For classification: majority vote
- For regression: average prediction



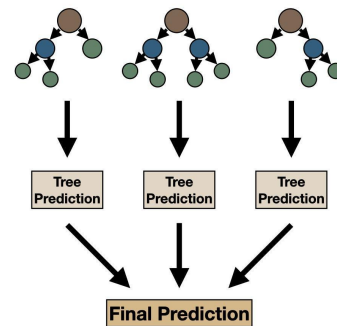
Why Not Just Use One Tree?

- A single decision tree is easy to interpret, but it's often unstable.
- Small changes in training data can produce a very different tree.
- Reduces Variance
- Lowers the instability of a single tree
- Averaging many trees gives more robust predictions
- Random feature selection reduces correlation among trees

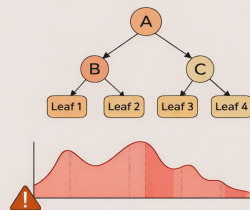
Single Decision Tree



Decision Tree Ensemble

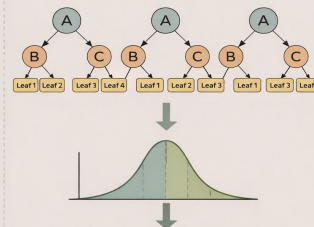


Single Decision Tree



- One decision tree on training data
- Predicted values can vary greatly with small changes in data

Random Forest



- ✓ Combines multiple decision trees' predictions
- ✓ Smoother, more stable predictions

How Random Forest Works

Phase I

- Start with original data set.

Phase II

- Create many resampled datasets with slight changes.

Phase III

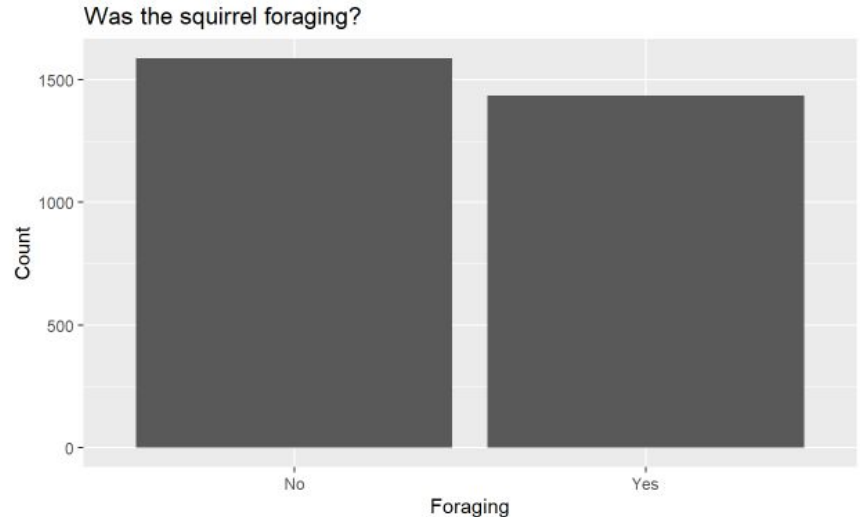
- Train one tree per sample using random features

Phase IV

- Output: Majority Vote or Average Prediction

Application: Central Park Squirrel Dataset

```
squirrels |>
  mutate(
    foraging_label = case_when(
      is.na(foraging) ~ "Missing",
      foraging == TRUE ~ "Yes",
      foraging == FALSE ~ "No"
    )
  ) |>
  count(foraging_label) |>
  ggplot(aes(x = foraging_label, y = n)) +
  geom_col() +
  labs(
    title = "Was the squirrel foraging?",
    x = "Foraging",
    y = "Count"
  )
```



- We choose a behavior target: foraging (Y/N)
- Balanced target: Yes 47.5% vs No 52.5%

Modeling Workflow

1. **Clean & recode** - convert Yes/No, handle unknown, parse number
2. **Split data** - 80/20 train-test split
3. **Preprocess recipe** - impute numeric + handle unknown/novel categories
4. **Tune RF** - cross-validate mtry and min_n
5. **Final fit + test** - train final model, evaluate on held-out test set
 - Final model: 500 trees

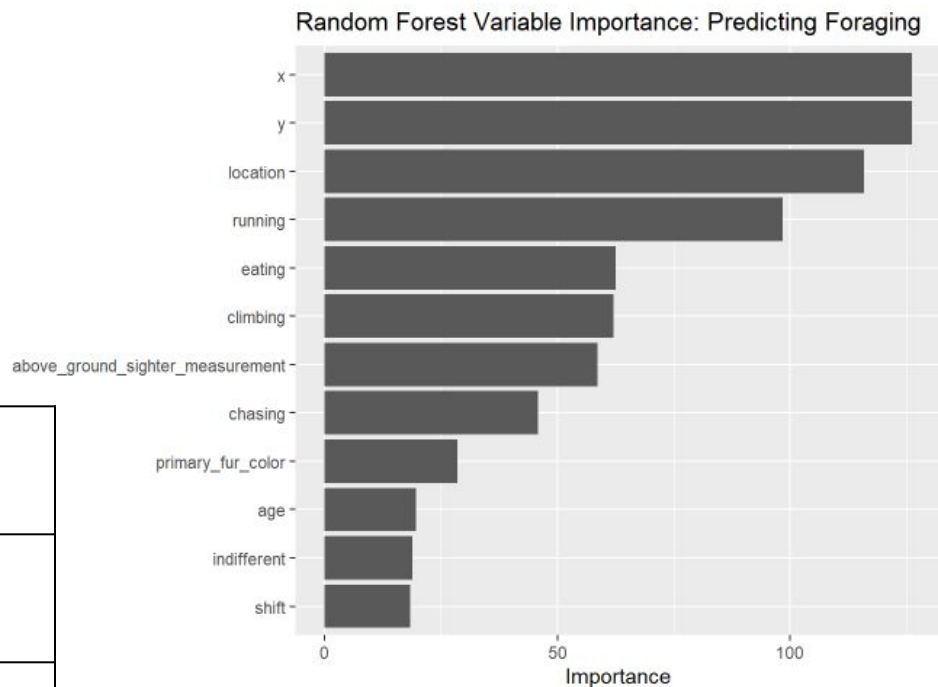
```
## == Workflow ==  
## Preprocessor: Recipe  
## Model: rand_forest()  
##  
## — Preprocessor —  
## 3 Recipe Steps  
##  
## • step_impute_median()  
## • step_unknown()  
## • step_novel()  
##  
## — Model —  
## Random Forest Model Specification (classification)  
##  
## Main Arguments:  
##   mtry = 3  
##   trees = 500  
##   min_n = 2  
##  
## Engine-Specific Arguments:  
##   importance = impurity  
##   probability = TRUE  
##  
## Computational engine: ranger
```


Results and Interpretation

Random Forest Confusion Matrix

	True: No	True: Yes
Pred: No	266	74
Pred: Yes	52	213

Model	Accuracy	Balance Accuracy	AUC
Random Forest	0.792	0.789	0.872
Single Tree	0.798	0.796	0.846



Boosting

Adaptive Boosting

$$err = \frac{1}{N} \sum_{i=1}^N I(y_i \neq G(x_i))$$

- Binary classifications
 - Response variable
 - $Y \in \{-1, 1\}$. Given predictor matrix, X , construct a classifier $G(X)$ that predicts 1 or -1.
- Builds classifiers $G_m(x)$, $m = 1, 2, \dots, M$ step by step
- At iteration m , selects the best classifier based on current weights
- Misclassified samples in step $m \rightarrow$ higher weight in $(m + 1)$
- Correctly classified samples $\rightarrow$ lower weight
- Each classifier $G_m(x)$ gets a stage weight based on its error
- Final prediction = weighted combination of all $G_m(x)$
- Classifier $G_m(x)$ returns discrete value, the AdaBoost.M1 algorithm is known as “Discrete AdaBoost”
 - You can revise the above algorithm if it returns continuous value,

Boosting

- How it works
 - Builds model **iteratively** by adding weak learners (e.g., trees)
 - Each step fits a model to the **current residuals (errors)**
 - Updates prediction by adding a **scaled contribution**
- **Tree depth** → controls model complexity
- **Number of iterations (trees)** → controls learning length
- **Learning rate (λ)**:
 - Shrinks each step's contribution
 - Values between **0 and 1**
 - Helps prevent overfitting
- Calculated across all trees in the ensemble
- Example:
 - Average **Gini improvement** per variable
- Similar approach to **bagging/random forests**

Stochastic Gradient Boosting

- Boosting can be viewed as a forward stagewise additive model
- It works by minimizing exponential loss
- Extended into methods like:
 - Real AdaBoost
 - Gentle AdaBoost
 - LogitBoost
- These are unified under Gradient Boosting Machines (GBM)
 - Response: $y \in \{-1, 1\}$
 - Model output: $f(x) \in (-\infty, \infty)$
 - Predicted probability:

$$\hat{p} = \frac{1}{1 + e^{-f(x)}}$$

Boosting Pros and Cons

- Pros

- High predictive accuracy
- Minimal preprocessing required
- Works well with tree-based models

- Cons

- Slower than single trees
- Less interpretable

Rule-Based Models

Rules Based Models

- How they work

- Learn patterns from data
- Convert patterns into logical rules
- Apply rules to classify or predict outcomes

- Advantages

- Easy to interpret
- Transparent decision making
- Often derived from decision trees

- Disadvantages

- Can create too many rules
- Overfit data
- Less accurate than more complex models

Cubist

Cubist Method Overview

- Based off M5 method created by Ross Quinlan
- Steps in Cubist Method:
 - a. Partition data based on specific rules (similar to other tree-based models)
 - b. Fits linear model for each node (similar to Regression Model Trees)
 - c. Committees are created, which is similar to boosting, where each new committee is built to correct the residuals of previous model.
 - d. Final prediction is average of predictions from all committees.
 - e. Use K-nearest neighbors from training dataset on final prediction to make adjustments.

Smoothing Process

- Linear models are generated at each node, including both internal and terminal nodes

$$\hat{y}_{par} = a \cdot \hat{y}_{(k)} + (1 - a) \cdot \hat{y}_{(p)}$$

where:

$\hat{y}_{par}$ = *smooted prediction*

$\hat{y}_{(k)}$ = *prediction from current model*

$\hat{y}_{(p)}$ = *prediction from parent model*

a = *smoothing coefficient*

$$a = \frac{Var(e_{(p)}) - Cov(e_{(k)}, e_{(p)})}{Var(e_{(p)} - e_{(k)})}$$

where:

$e_{(p)}$ = *collection of residuals for parent model*

$e_{(k)}$ = *collection of residuals for child model*

Boosting vs. Committees

Boosting

- Generates initial model, calculates residuals, and builds subsequent models on residuals
- Final model is the sum of these models

Committees

- Generates initial model, calculates residuals, takes difference of actual values and residuals, generates model off modified values
- Final model is average of these committees

$$y_m^* = y - (\hat{y}_{(m-1)} - y)$$

y_m^* = *adjusted target*

y = *observed value of training data*

$\hat{y}_{(m-1)}$ = *predicted value of previous committee*

K Nearest Neighbors

- Given a sample you are trying to predict, find k nearest neighbors within training set based on predictors
- Original method describes use of Manhattan method for determining k nearest neighbors, but Euclidean method is currently more common

$$\textit{Manhattan: } D(x, y) = \sum |x_i - y_i|$$

$$\textit{Euclidean: } D(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

K Nearest Neighbors (cont.)

- Calculate residuals for k nearest neighbors based on what model predicts and their actual output
- Average the residuals for k nearest neighbors
- Add average of residuals to model prediction of sample to get final prediction

$$\hat{y}_{final} = \frac{1}{K} \sum_{l=1}^K w_l \cdot [t_l + (\hat{y} - \hat{t}_l)]$$

where:

K = # of neighbors

w_l = weights

t_l = observed outcome of training neighbor

$\hat{y}$ = model prediction before KNN adjustment

$\hat{t}_l$ = model prediction of training neighbor

$$w_l = \frac{1}{D_l + 0.5}$$

where:

D_l = distance of training neighbor to prediction sample, normalized to 1

Examples

Datasets Used

Categorical

- Squirrel dataset from 2018 Central Park Census
 - Contains information such as location, color, and activity

Numerical

- Tree dataset from U.S. Department of Agriculture
 - Data from 1992-2012 within 17 cities within U.S.
 - Contains information such as region, tree type, size, and age
 - All examples will be used to predict age of tree

Basic Tree Example

```
```{r}
Create Model on Training Set
tree_model = rpart(Age ~ ., data = train)

Predict on Testing Set using Model
pred_tree = predict(tree_model, test)
```
```

control = rpart.control(maxdepth = n)
can be added to set maximum depth



```
```{r}
tree_values = data.frame(obs = test$Age, pred = pred_tree)
defaultSummary(tree_values)
```
```

| RMSE | Rsquared | MAE |
|------------|-----------|-----------|
| 12.3352291 | 0.7008752 | 8.9285506 |

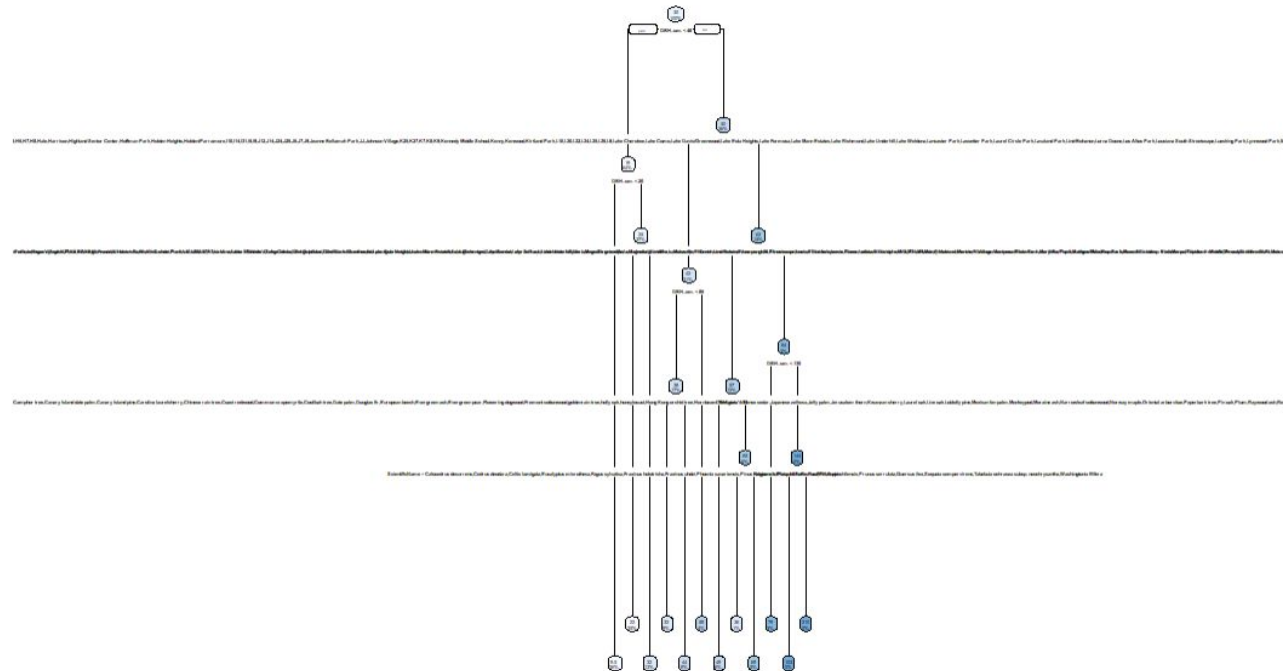
Useful R Functions

```
# Generates Dataframe of all Rules  
rpart.rules(tree_model)
```

| | Age
<chr> | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> | <chr> |
|----|--------------|-------|----------|-------|-------|-------|-------|-------|-------|
| 4 | 9.5 | when | DBH..cm. | < | 20 | | | | |
| 10 | 22.0 | when | DBH..cm. | is | 20 | to | 48 | & | Zone |
| 11 | 31.7 | when | DBH..cm. | is | 20 | to | 48 | & | Zone |
| 24 | 32.2 | when | DBH..cm. | is | 48 | to | 86 | & | Zone |
| 58 | 36.1 | when | DBH..cm. | >= | | | 73 | & | Zone |
| 25 | 44.3 | when | DBH..cm. | is | 48 | to | 86 | & | Zone |
| 28 | 48.9 | when | DBH..cm. | is | 48 | to | 73 | & | Zone |
| 13 | 59.0 | when | DBH..cm. | >= | | | 86 | & | Zone |
| 59 | 68.8 | when | DBH..cm. | >= | | | 73 | & | Zone |
| 30 | 78.6 | when | DBH..cm. | is | 48 | to | 130 | & | Zone |

Useful R Functions (cont.)

```
# Generates Tree Visualization
rpart.plot(tree_model)
```



Regression Tree Example

```
## Regression Model Tree
```

```
```{r}
```

```
Create Model on Training Set
```

```
reg_tree_model = M5P(Age ~ ., data = train)
```

```
Predict on Testing Set using Model
```

```
reg_pred_tree <- predict(reg_tree_model, test)
```

```
```
```

```
```{r}
```

```
reg_tree_values = data.frame(obs = test$Age, pred = reg_pred_tree)
```

```
defaultSummary(reg_tree_values)
```

```
```
```

Can be modified with: `Weka_control()`

https://search.r-project.org/CRAN/refmans/RWeka/html/Weka_control.html

| RMSE | Rsquared | MAE |
|-----------|----------|----------|
| 12.080477 | 0.725763 | 8.316904 |

Rule-Based Tree Example

```
## Rule-Based Model Tree
```

```
```{r}
```

```
Create Model on Training Set
```

```
rule_tree_model = M5Rules(Age ~ ., data = train)
```

```
Predict on Testing Set using Model
```

```
rule_pred_tree <- predict(rule_tree_model, test)
```

```
```
```

Can be modified with: *Weka_control()*
https://search.r-project.org/CRAN/refmans/RWeka/html/Weka_control.html

```
```{r}
```

```
rule_tree_values = data.frame(obs = test$Age, pred = rule_pred_tree)
```

```
defaultSummary(rule_tree_values)
```

```
```
```

| RMSE | Rsquared | MAE |
|-----------|----------|----------|
| 13.434916 | 0.671929 | 8.760404 |

Bagged Tree Example

```
## Bagged Trees
```

```
```{r}
```

```
Create Model on Training Set
```

```
bag_tree_model = bagging(Age ~ ., data = train)
```

```
Predict on Testing Set using Model
```

```
bag_pred_tree <- predict(bag_tree_model, test)
```

```
```
```

Modify with *control = rpart.control()*
nbagg = n to set number of tree ensembles



```
```{r}
```

```
bag_tree_values = data.frame(obs = test$Age, pred = bag_pred_tree)
```

```
defaultSummary(bag_tree_values)
```

```
```
```

| RMSE | Rsquared | MAE |
|------------|-----------|-----------|
| 12.4304156 | 0.6981066 | 8.9177694 |

Random Forest

```
## Random Forest
```

```
```{r}
```

```
Redefine Training and Test Sets
```

```
rf_train = train |> select(-Zone, -SpCode, -ScientificName, -CommonName)
```

```
rf_test = test |> select(-Zone, -SpCode, -ScientificName, -CommonName)
```

```
Create Model on New Training Set
```

```
rf_tree_model = randomForest(Age ~ ., data = rf_train, na.action = na.omit)
```

```
Predict on New Testing Set using Model
```

```
rf_pred_tree <- predict(rf_tree_model, rf_test)
```

Need to include *na.action*

```
```{r}
```

```
rf_tree_values = data.frame(obs = rf_test$Age, pred = rf_pred_tree)
```

```
defaultSummary(rf_tree_values)
```

RMSE	Rsquared	MAE
10.876564	0.798526	7.163564

Boosting Example

```
## Boosting
```{r}
Create Model on Training Set
boost_tree_model = gbm(Age ~ ., data = train, distribution = "gaussian")

Predict on Testing Set using Model
boost_pred_tree <- predict(boost_tree_model, test)
```
```



Use *n.trees* = *n* to set
number of trees

Using 100 trees...

```
```{r}
boost_tree_values = data.frame(obs = test$Age, pred = boost_pred_tree)
defaultSummary(boost_tree_values)
```
```

| RMSE | Rsquared | MAE |
|------------|-----------|-----------|
| 11.0674286 | 0.7643727 | 7.9161944 |

Cubist Example

```
## Cubist Tree
```{r}
Redefine Training and Test Sets
cubist_train = train |> select(-Age, -Zone, -ScientificName, -CommonName)
cubist_test = test |> select(-Age, -Zone, -ScientificName, -CommonName)
cubist_train_age = train$Age
cubist_test_age = test$Age

names(cubist_train) = make.names(names(cubist_train))

Create Model on New Training Set
cubist_model = cubist(cubist_train, cubist_train_age, committees = 10)

Predict on New Testing Set using Model
cubist_pred <- predict(cubist_model, cubist_test)
```

```{r}
cubist_values = data.frame(obs = cubist_test_age, pred = cubist_pred)
defaultSummary(cubist_values)
```
```

| RMSE | Rsquared | MAE |
|-----------|-----------|-----------|
| 9.1356521 | 0.8368976 | 5.8868152 |

Coefficients of Linear Models for Cubist Method

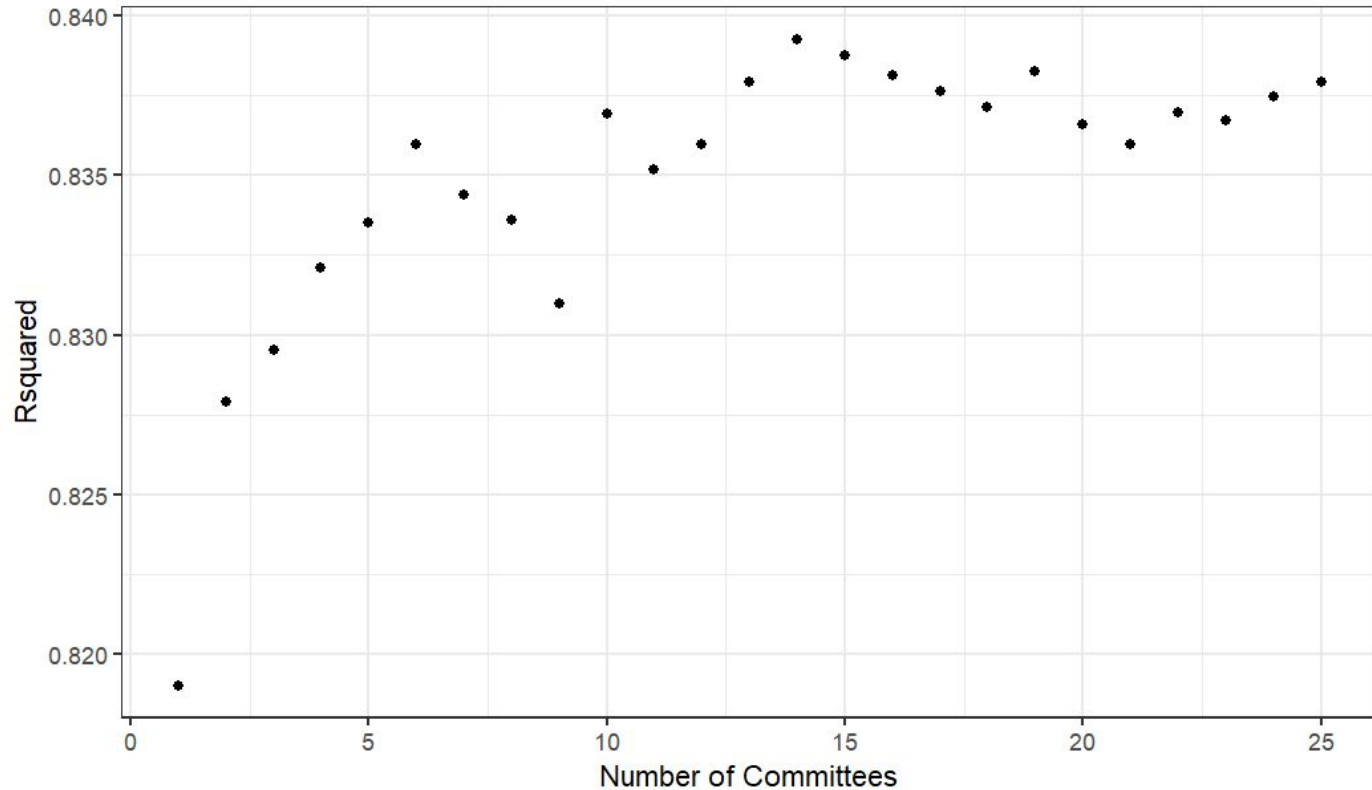
```
> cubist_model$coefficients |> head(3)
```

| | (Intercept) | Region | Park.Street | SpCode | TreeType | DBH..cm. | TreeHt..m. | CrnBase | | |
|----|-------------|-------------|--------------|-------------|-----------|----------|------------|---------|--|--|
| 1 | -1.2 | NA | NA | NA | NA | 0.554 | 0.19 | -0.2 | | |
| 15 | -4.2 | NA | NA | NA | NA | 1.702 | 5.31 | -5.1 | | |
| 29 | -1.8 | NA | NA | NA | NA | 1.110 | 0.35 | -0.3 | | |
| | CrnHt..m. | CdiaPar..m. | CDiaPerp..m. | AvgCdia..m. | Leaf..m2. | dbh1 | committee | rule | | |
| 1 | -0.21 | NA | NA | 0.12 | NA | 0.01 | 1 | 1 | | |
| 15 | -6.81 | 0.68 | 0.63 | 0.32 | NA | NA | 1 | 2 | | |
| 29 | -0.38 | NA | NA | NA | NA | NA | 1 | 3 | | |

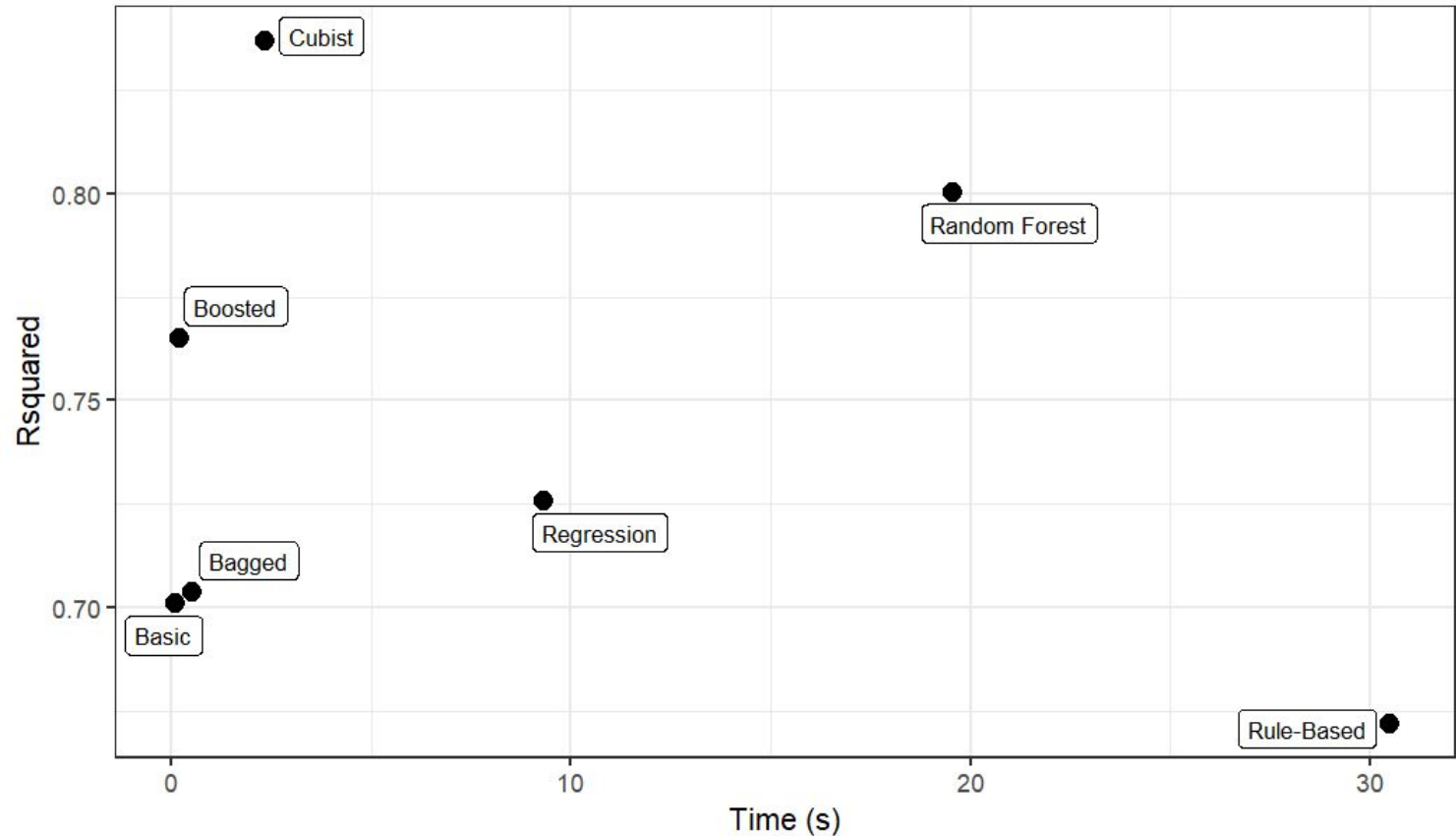
```
> cubist_model$coefficients |> tail(3)
```

| | (Intercept) | Region | Park.Street | SpCode | TreeType | DBH..cm. | TreeHt..m. | CrnBase | | |
|------|-------------|-------------|--------------|-------------|-----------|----------|------------|---------|--|--|
| 7043 | 98.1 | NA | NA | NA | NA | -0.256 | -0.36 | -1.0 | | |
| 7057 | 35.4 | NA | NA | NA | NA | 0.352 | -0.64 | 1.5 | | |
| 7071 | -375.0 | NA | NA | NA | NA | 4.220 | NA | NA | | |
| | CrnHt..m. | CdiaPar..m. | CDiaPerp..m. | AvgCdia..m. | Leaf..m2. | dbh1 | committee | rule | | |
| 7043 | 0.04 | NA | NA | NA | NA | 0.125 | 10 | 50 | | |
| 7057 | 0.64 | -1.16 | -0.83 | 1.68 | NA | 0.223 | 10 | 51 | | |
| 7071 | NA | NA | NA | NA | NA | NA | 10 | 52 | | |

Comparing Performance of Cubist Against Number of Committees



Comparing Performance of All Tree Models



Citations

Aha, D. W., Kibler, D., & Albert, M. K. (1991). *Instance-based learning algorithms*. **Machine Learning**, 6(1), 37–66. <https://doi.org/10.1007/BF00153759>

Kuhn, M., Weston, S., Keefer, C., & Coulter, N. (2024). *Cubist: Rule- and instance-based regression modeling* (Version 0.4.5). Comprehensive R Archive Network (CRAN). <https://cran.r-project.org/web/packages/Cubist/index.html>

Reference

- <https://scientistcafe.com/ids/splitting-criteria>
- Kuhn & Johnson (2013). *Applied Predictive Modeling*
- <https://odsc.medium.com/balancing-interpretability-and-predictive-power-with-cubist-models-in-r-858d2c936b79>

Sources

<https://www.geeksforgeeks.org/r-language/random-forest-approach-in-r-programming/>

<https://medium.com/@mohammedsaimquadri/random-forest-the-power-of-many-trees-4a619f3928b7>